

PLACEMENT PORTAL APPLICATION (MAD 1)

(Project Report)

1. Student Details

Name: Naina Bhatt

Roll Number: 24f3001863

Email: 24f3001863@ds.study.iitm.ac.in

About Me:

I am a student at IIT Madras BS Degree in Data Science and Applications program with an interest in tech, design, and multimedia. This was my first application development project and I enjoyed it thoroughly. I aim and hope to keep learning & building more and more applications and websites as the degree progresses.

2. Project Details

Project Title: Placement Portal Application

Problem Statement:

To build a web-based placement portal application that enables efficient interaction between students, companies, and administrators. The system should allow companies to post job opportunities, students to apply for jobs, and administrators to manage the entire placement process.

Approach:

The application was built using Flask, a Python framework, as the backend framework with SQLite for database & SQLAlchemy for database management. It is following a modular structure with separate models, templates, and routes. The system supports multiple user roles (Admin, Company, Student) and provides features like job postings, applications, and placement status tracking.

3. AI/LLM Declaration

I used ChatGPT Go and Gemini Pro to assist in SQLAlchemy model definitions, understanding Flask concepts, some minor debugging & improving code structure. The usage was approximately **20–25%**, mainly for guidance and minor code suggestions. All core logic, implementation, and integration were done manually.

4. Technologies and Frameworks Used

Technology	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
SQLite	Lightweight local database for storing user data
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap / CSS	Frontend styling and responsive UI mainly (minimal CSS)
Flask-Login	Authentication system for security

5. Database Schema / ER Diagram

Tables:

1. **User** - Stores common user details (id, name, email, password, role)
2. **Admin** - Stores predefined admin details (id, user_id)
3. **Company** - Stores additional company details (id, user_id, hr_contact, industry, website, is_approved, is_blacklisted)
4. **Student** - Stores additional student details (id, user_id, contact, education, skills, resume_path, is_blacklisted)
5. **PlacementDrive (Jobs)** - Stores details about placement drives created by a company (id, company_id, job_title, job_description, skills_required, salary_range, deadline, status)
6. **Application** - Stores details of student applying for a particular placement drive (id, student_id, drive_id, application_date, status)

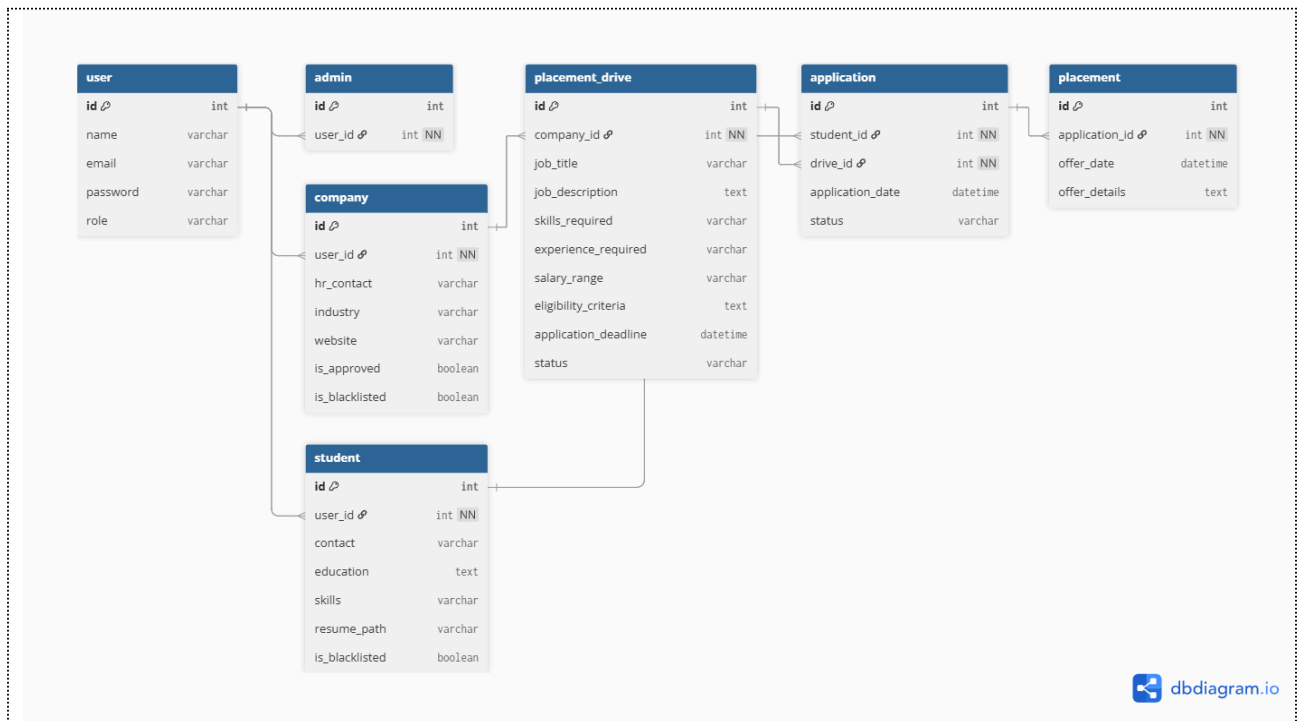
7. **Placement** - Stores records of hired/placed students (id, application_id, offer_date, offer_details)
-

Relationships:

- User → Admin / Company / Student (One-to-One)
- Company → PlacementDrive (One-to-Many)
- Student → Application (One-to-Many)
- PlacementDrive → Application (One-to-Many)
- Application → Placement (One-to-One)

One extra relationship is:

- Student ↔ PlacementDrive (Many-to-Many, implemented via Application table)



(This ER diagram was made by dbdiagram.io)

6. API Resource Endpoints

(Based on your Flask routes and functionality)

ENDPOINT	METHOD	DESCRIPTION
/login	POST	Authenticate user
/register	POST	Register new user
/student/dashboard	GET	Student dashboard
/company/dashboard	GET	Company dashboard
/admin/dashboard	GET	Admin dashboard
/apply/<drive_id>	POST	Apply for job
/post_drive	POST	Company posts job

7. Architecture and Features

Architecture Overview:

- **app.py** → Main Flask application entry point
- **models.py** → Database models using SQLAlchemy
- **templates/** → HTML pages (student, company, admin dashboards & additional pages)
- **static/** → Minimal CSS styling

Implemented Functionalities for Users:

Student:

- Can register & login
- Can view approved job postings
- Can apply for jobs
- Can view & track applications
- Can edit and update their profile

Company:

- Can register and login after admin approval
- Can post placement drives / job postings
- Can edit, remove or close placement drives
- Can view student applications
- Can shortlist students and update their selection status
- Can manage job listings

Admin:

- Can view all students, companies, applications, & drives
 - Can approve or reject company registrations
 - Can approve or reject placement drives created by companies
 - Can search students by name or contact information
 - Can search companies by name
 - Can blacklist or deactivate students and company accounts
 - Monitor platform activity
-

Other Functionalities

- Role-based authentication for all three users
 - Role-based dashboards for all three users
 - Resume upload and update option for students
 - Maintain complete application history for each student
 - Placement record management
-

8. Video Presentation

Drive Link to the video presentation of the application

 **Placement Portal Application (MAD 1) Video presentation.mp4**